SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO4 – ASSIGNMENT (PART 2)

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: CSA0601

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

TASK 1 – BUILD AND VALIDATE A DYNAMIC PROGRAMMING SOLUTION

Word Wrap as a Document-Layout Engine

1. Introduction

Word Wrap is an optimisation problem used in document-processing systems, text editors and word processors. The objective is to arrange a sequence of words into lines without exceeding a specified maximum line width.

If a line contains fewer characters than the maximum width, the remaining positions are unused spaces. These unused spaces create a penalty for every non-final line. The final line does not contribute any penalty.

A simple greedy approach may place as many words as possible on each line. However, this may not produce the minimum total penalty. Therefore, Dynamic Programming is used to evaluate different possible line arrangements and identify the arrangement with the minimum total cost.

1.1 Implement the Solution

Problem definition

Consider the following example:

```
Words:
Dynamic Programming provides efficient solutions
to complex optimization problems
```

Assume the maximum line width is 20 characters.

The algorithm must:

1. Arrange all words in their original order.
2. Ensure that no line exceeds the maximum width.
3. Calculate unused spaces on each line.
4. Calculate the penalty of each non-final line.
5. Assign zero penalty to the final line.
6. Select the arrangement with minimum total penalty.

Dynamic Programming State

Let:

$$DP[i] = \text{Minimum penalty required to arrange words from position } i \text{ to } n$$

Here, i represents the starting index of the remaining words.

The base condition is:

$$DP[n] = 0$$

because no words remain after the final position.

If words from position i to j fit in one line, the unused spaces are:

$$Spaces = Width - CharactersUsed$$

For a non-final line, the penalty is:

$$LinePenalty = Spaces^2$$

The transition is:

$$DP[i] = \min_{j \geq i}(LinePenalty(i, j) + DP[j + 1]))$$

The final line is treated specially and receives zero penalty.

```python
def word_wrap(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    next_word = [-1] * n
    dp[n] = 0
    # Calculate minimum penalty from right to left
    for i in range(n - 1, -1, -1):
        characters_used = 0
        for j in range(i, n):
            if j == i:
                characters_used = len(words[j])
            else:
                characters_used += 1 + len(words[j])
            if characters_used > width:
                break
            unused_spaces = width - characters_used
            # Final line has no penalty
            if j == n - 1:
                line_penalty = 0
            else:
                line_penalty = unused_spaces * unused_spaces
            total_cost = line_penalty + dp[j + 1]
            if total_cost < dp[i]:
                dp[i] = total_cost
                next_word[i] = j + 1
    # Reconstruct the arrangement
    lines = []
    line_details = []
    i = 0
    while i < n:
        j = next_word[i]
        if j == -1:
            raise ValueError("A word is longer than the line width.")
        selected_words = words[i:j]
        line_text = " ".join(selected_words)
        characters_used = len(line_text)
        unused_spaces = width - characters_used
        if j == n:
            line_penalty = 0
        else:
            line_penalty = unused_spaces * unused_spaces
        lines.append(line_text)
        line_details.append({
            "words": selected_words,
            "used": characters_used,
            "unused": unused_spaces,
```

```
        })
        i = j
    return dp[0], lines, line_details, dp
# Main program
words = [
    "Dynamic",
    "Programming",
    "provides",
    "efficient",
    "solutions",
    "to",
    "complex",
    "optimization",
    "problems"
]
width = 20
minimum_penalty, lines, details, dp = word_wrap(words, width)
print("WORD WRAP RESULT")
print("----------------")
for index, detail in enumerate(details):
    print("\nLine", index + 1)
    print("Words:", " ".join(detail["words"]))
    print("Unused spaces:", detail["unused"])
    print("Line penalty:", detail["penalty"])
print("\nMinimum Total Penalty:", minimum_penalty)
print("\nDP STATES")
for i in range(len(dp)):
    print("DP[", i, "] =", dp[i])
```

Output

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ASUS/Downloads/1.py ==========
WORD WRAP RESULT
----------------

Line 1
Words: Dynamic Programming
Unused spaces: 1
Line penalty: 1

Line 2
Words: provides efficient
Unused spaces: 2
Line penalty: 4

Line 3
Words: solutions to
Unused spaces: 8
Line penalty: 64

Line 4
Words: complex optimization
Unused spaces: 0
Line penalty: 0

Line 5
Words: problems
Unused spaces: 12
Line penalty: 0

Minimum Total Penalty: 69

DP STATES
DP[ 0 ] = 69
DP[ 1 ] = 149
DP[ 2 ] = 68
DP[ 3 ] = 165
DP[ 4 ] = 64
DP[ 5 ] = 164
DP[ 6 ] = 0
DP[ 7 ] = 64
DP[ 8 ] = 0
DP[ 9 ] = 0
```

Explanation of the Program

1. The input words are stored in a list.
2. The dp array stores the minimum penalty for arranging the remaining words.
3. The next_word array stores the position where the next line begins.
4. The algorithm examines different groups of consecutive words.
5. If a group fits within the maximum width, its penalty is calculated.
6. The algorithm adds the penalty to the best cost of arranging the remaining words.
7. The minimum result is stored in the DP array.
8. The next_word array is used to reconstruct the actual line arrangement.
9. Finally, the program displays every line, unused spaces, individual penalties and the minimum total penalty.

Time Complexity

There are n possible starting positions. For each starting position, the algorithm examines at most n possible ending positions.

$$\boxed{O(n^2)}$$

The DP and reconstruction arrays require:

$$\boxed{O(n)}$$

auxiliary space, excluding the input and output strings.

1.2 Expose the DP Behaviour

The DP array shows how the algorithm constructs the optimum solution from smaller subproblems.

DP State	Meaning
DP[0]	Minimum penalty for arranging all words
DP[1]	Minimum penalty starting from the second word
DP[2]	Minimum penalty starting from the third word
DP[i]	Minimum penalty for all words from index i onwards
DP[n]	Zero penalty because no words remain

The algorithm processes the states from right to left. This is necessary because the calculation of DP[i] uses the previously calculated value DP[j+1].

For example, if the algorithm considers placing words 1 to 3 on one line, it calculates the unused spaces and adds the result to the minimum penalty of arranging the remaining words from position 4.

This demonstrates the principle of optimal substructure: an optimal arrangement of the complete text can be constructed using optimal arrangements of the remaining words.

1.3 Requirement Change

The same text can be tested with three different line widths:

- Width 15
- Width 20
- Width 30

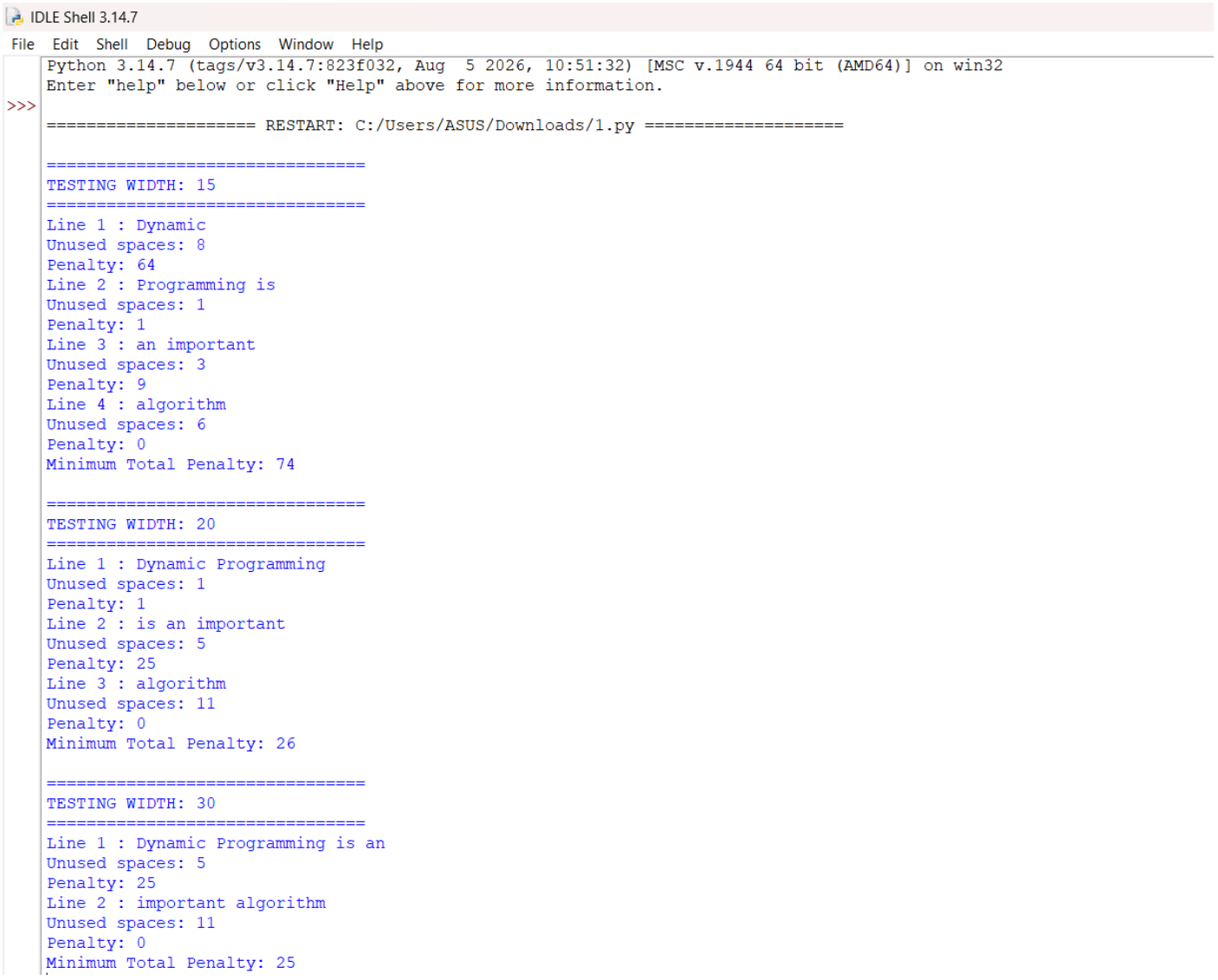
Predictions before execution

Line Width	Prediction of Line Arrangement	Expected Penalty Behaviour
15	More lines will be required. Some long words may need to be placed individually.	Penalty may increase because more lines and unused spaces may be produced.
20	The arrangement will be more balanced.	Expected to be lower than some narrower arrangements.
30	More words may fit into each line. The first change is expected near the beginning of the text.	The penalty is expected to decrease in many cases, but it is not guaranteed to decrease for every wider width.

Program for testing multiple widths

```
def word_wrap(words, width):
    n = len(words)
    dp = [0] * (n + 1)
    spaces = [0] * n
    penalty = [0] * n
    result = [0] * n
    for i in range(n - 1, -1, -1):
        dp[i] = float('inf')
        length = 0
        for j in range(i, n):
            length += len(words[j])
            if length + (j - i) > width:
                break
            unused = width - (length + (j - i))
            cost = unused ** 2 if j != n - 1 else 0
            if cost + dp[j + 1] < dp[i]:
                dp[i] = cost + dp[j + 1]
                result[i] = j + 1
        if result[i] == 0:
            raise ValueError("Word cannot fit within the given width")
    lines = []
    details = []
    i = 0
    while i < n:
        j = result[i]
        line = " ".join(words[i:j])
        unused = width - len(line)
        cost = unused ** 2 if j != n else 0
        lines.append(line)
        details.append({"unused": unused, "penalty": cost})
        i = j
    return dp[0], lines, details, dp
# Input words
words = ["Dynamic", "Programming", "is", "an", "important", "algorithm"]
# Test different widths
test_widths = [15, 20, 30]
for width in test_widths:
    print("\n================================")
    print("TESTING WIDTH:", width)
    print("================================")
    try:
        minimum_penalty, lines, details, dp = word_wrap(words, width)
        for i in range(len(lines)):
            print("Line", i + 1, ":", lines[i])
            print("Unused spaces:", details[i]["unused"])
            print("Penalty:", details[i]["penalty"])
        print("Minimum Total Penalty:", minimum_penalty)
```

Output



Comparison and Interpretation

A smaller line width restricts the number of words that can be placed together. A larger line width provides more possible combinations. However, the minimum penalty does not necessarily decrease monotonically with every increase in width, because the objective uses squared unused spaces and the final line is free of penalty.

The actual program output should be compared with the predictions. Any difference between the predicted arrangement and the actual arrangement should be explained using the DP states.

1.4 Boundary Test

A boundary test is required to verify whether the algorithm handles special inputs correctly.

Selected boundary case: A word whose length is exactly equal to the maximum line width.

Example:

```
Words = ["Algorithm", "Design"]
Width = 9
```

The word "Algorithm" contains exactly 9 characters. It should fit into the first line without exceeding the permitted width.

Expected arrangement:

```
Algorithm
Design
```

The first line contains zero unused spaces, and its penalty is:

$$(9-9)^2 = 0$$

The final line has no penalty according to the problem requirement.

Additional boundary cases

Test Case	Input	Expected Behaviour
Exact width	One word has length equal to width	Word must be accepted
Word too long	A word is longer than width	Program should report an error
One word	Only one word is supplied	One final line with zero penalty
Empty input	No words are supplied	Minimum penalty should be zero

Conclusion of Task 1

The Word Wrap problem is successfully modelled using Dynamic Programming. The algorithm evaluates possible line arrangements, stores minimum penalties and reconstructs the optimal arrangement. Testing different widths demonstrates how a changed requirement affects the final layout. The boundary test checks whether the implementation handles exact-width words correctly.

TASK 2 – GREEDY VS DYNAMIC PROGRAMMING

Build It, Break It, Explain It

2. Introduction

The Knapsack problem involves selecting items to maximise total value without exceeding a given capacity. Each item has:

- Item ID
- Weight
- Value

Two different formulations are implemented:

➢ **Solution A – Fractional Knapsack**

Items can be divided into fractions. A greedy strategy is used to select items based on their value-to-weight ratio.

➢ **Solution B – 0/1 Knapsack**

Items cannot be divided. Each item must either be selected completely or rejected. Dynamic Programming is used to find the optimal combination.

2.1 Normal Case

Dataset

Assume the vehicle capacity is 50 units.

Item	Weight	Value	Value-to-Weight Ratio
A	10	60	6.0
B	20	100	5.0
C	30	120	4.0

➢ **Solution A – Fractional Knapsack**

The items are sorted by descending value-to-weight ratio.

The algorithm first selects A, then B, and finally a fraction of C.

Step	Item	Weight Loaded	Fraction	Value Added	Remaining Capacity
1	A	10	1.00	60	40
2	B	20	1.00	100	20
3	C	20	0.67	80	0

Total value:

$$60 + 100 + 80 = \boxed{240}$$

➢ **Solution B – 0/1 Knapsack**

In the 0/1 formulation, an item cannot be divided. The possible optimal selection is A and B.

Selected Item	Weight	Value
A	10	60
B	20	100
Total	30	160

The remaining capacity is 20, but item C weighs 30 and cannot fit.

The 0/1 optimum is 160 for this dataset.

Python Implementation – Both Solutions

```
def fractional_knapsack(items, capacity):
    items = sorted(
        items,
        key=lambda item: item[2] / item[1],
        reverse=True
    )
    remaining_capacity = capacity
    total_value = 0
    selected_items = []
    print("\nFRACTIONAL KNAPSACK")
    print("--------------------")
    for item_id, weight, value in items:
        if remaining_capacity == 0:
            break
        if weight <= remaining_capacity:
            fraction = 1
            loaded_weight = weight
            added_value = value
        else:
            fraction = remaining_capacity / weight
            loaded_weight = remaining_capacity
            added_value = fraction * value
        remaining_capacity = remaining_capacity - loaded_weight
        total_value = total_value + added_value
        selected_items.append(
            (item_id, fraction, loaded_weight, added_value)
        )
        print("Item:", item_id)
        print("Fraction:", round(fraction, 2))
        print("Loaded Weight:", loaded_weight)
        print("Value Added:", round(added_value, 2))
        print("Remaining Capacity:", remaining_capacity)
    print("Selected Items:", selected_items)
    print("Maximum Value:", total_value)
    return total_value
def zero_one_knapsack(items, capacity):
    n = len(items)
    dp = [[0 for j in range(capacity + 1)]
          for i in range(n + 1)]
    operations = 0
    for i in range(1, n + 1):
        item_id, weight, value = items[i - 1]
        for w in range(capacity + 1):
            operations += 1
            if weight <= w:
                include_value = value + dp[i - 1][w - weight]
                exclude_value = dp[i - 1][w]
```

Output

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ASUS/Downloads/1.py ==========

FRACTIONAL KNAPSACK
--------------------
Item: A
Fraction: 1
Loaded Weight: 10
Value Added: 60
Remaining Capacity: 40
Item: B
Fraction: 1
Loaded Weight: 20
Value Added: 100
Remaining Capacity: 20
Item: C
Fraction: 0.67
Loaded Weight: 20
Value Added: 80.0
Remaining Capacity: 0
Selected Items: [('A', 1, 10, 60), ('B', 1, 20, 100), ('C', 0.6666666666666666, 20, 80.0)]
Maximum Value: 240.0

0/1 KNAPSACK
------------
Selected Items: ['B', 'C']
Maximum Value: 220
DP Operations: 153
```

Interpretation

The fractional version can use the remaining 20 units by loading part of item C. The 0/1 version cannot divide C, so it selects only complete items.

The two algorithms solve different formulations of the Knapsack problem. The fractional greedy algorithm is optimal for the fractional formulation, while the DP algorithm is optimal for the 0/1 formulation.

2.2 Break the Greedy Intuition

A greedy algorithm that selects the highest value-to-weight ratio first is not guaranteed to produce an optimal solution for 0/1 Knapsack.

To demonstrate this, construct the following dataset.

Counterexample Dataset

Vehicle capacity = 50 units.

Item	Weight	Value	Value-to-Weight Ratio
A	10	60	6.0
B	20	100	5.0
C	30	120	4.0

The greedy algorithm selects A first because it has the highest ratio.

It then selects B because B also fits in the remaining capacity.

After selecting A and B:

- Total weight = 30
- Total value = 160
- Remaining capacity = 20

Item C cannot fit.

Therefore, the greedy result is:

$$\boxed{160}$$

However, selecting B and C gives:

- Total weight = 20 + 30 = 50
- Total value = 100 + 120 = 220

Thus, the optimal result is:

$$\boxed{220}$$

➢ **First Greedy Commitment That Prevents the Optimum**

The first commitment is selecting item A. Although A has the highest value-to-weight ratio, selecting it consumes 10 units of capacity. This prevents the combination of B and C from being selected together.

The greedy solution is therefore suboptimal for the 0/1 formulation.

Comparison

Method	Selected Items	Total Weight	Total Value
Greedy ratio-based	A, B	30	160
Dynamic Programming	B, C	50	220

Difference in Solution Quality

$$220 - 160 = 60$$

The greedy method produces a value that is 60 units lower than the DP optimum.

2.3 Engineering Comparison

The two implementations can be compared using computational work, memory, solution quality and optimality guarantee.

Factor	Fractional Knapsack	0/1 Knapsack
Strategy	Greedy	Dynamic Programming
Selection	Complete items or fractions	Complete items only
Main decision	Highest value-to-weight ratio	Include or exclude each item
Sorting	Required by ratio	Not required
Computational work	Sorting dominates	DP table calculation
Time complexity	\(O(n\log n)\)	\(O(nW)\)
Memory requirement	\(O(1)\) additional space, excluding sorting/output	\(O(nW)\) for the full DP table
Solution quality	Optimal for fractional formulation	Optimal for 0/1 formulation
Optimality guarantee	Yes, for fractional Knapsack	Yes, for 0/1 Knapsack

Here, n is the number of items and W is the integer capacity.

Operation Evidence

For the 0/1 implementation, the number of DP cell calculations is:

$$n(W + 1)$$

For the example:

$$3(50 + 1) = 153$$

Thus, the implementation evaluates 153 DP cells.

The fractional implementation sorts three items and then examines them in order. It performs far fewer selection operations for this small example.

However, the two algorithms must not be judged solely by their operation counts because they solve different versions of the problem.

2.4 Changed Requirement

Management now states that the result must be generated extremely quickly and that a near-optimal result is acceptable.

This changes the engineering decision. The system no longer requires an exact optimum in every case. The greedy method can be considered when rapid decision-making is more important than guaranteeing the best 0/1 combination. It sorts the items by ratio and makes selections in a single pass after sorting.

For the counterexample, the greedy method produces 160 while Dynamic Programming produces 220. Therefore, the greedy result is not acceptable if the company requires a result close to the optimum for this input.

A practical system can use the following approach:

1. Use the greedy method to obtain a quick initial solution.
2. Calculate its total value and weight.
3. If time permits, run an improvement method or an exact DP solution for smaller instances.
4. Compare the improved solution with the greedy result.
5. Return the best solution available within the time limit.

Conclusion of Task 2

Fractional Knapsack and 0/1 Knapsack require different algorithmic approaches. Greedy selection based on value-to-weight ratio is optimal when fractions are allowed. However, it can fail when items are indivisible. Dynamic Programming evaluates combinations and guarantees the optimal 0/1 solution, but its computational and memory requirements increase with capacity. The implementation demonstrates why algorithm selection must depend on the actual business requirement.

TASK 3 – ROUTING ENGINE UNDER CHANGING NETWORK CONDITIONS

Dijkstra, Bellman-Ford, Floyd and Warshall

3. Introduction

A logistics platform uses a weighted directed graph to represent its transportation network. Vertices represent locations, and edges represent travel connections. The edge weights represent travel costs or distances.

The routing engine must find shortest paths, handle negative-weight edges, detect negative cycles and calculate reachability between every pair of vertices.

The following algorithms are implemented:

1. Dijkstra's Algorithm
2. Bellman-Ford Algorithm
3. Floyd's Algorithm
4. Warshall's Algorithm

Graph Definition

The network contains eight vertices and twelve directed edges.

Vertices:

$$V = \{A, B, C, D, E, F, G, H\}$$

Edge	Weight
A → B	4
A → C	5
B → C	1
B → D	6
C → D	2
C → E	5
D → E	1
D → F	4
E → F	2
E → G	6
F → G	1
G → H	3

This graph is used for normal routing. All edge weights are non-negative, so Dijkstra's algorithm is valid.

- **Stage A – Normal Routing: Dijkstra's Algorithm**

Algorithm Explanation

Dijkstra's algorithm calculates the shortest distance from a selected source vertex to all other reachable vertices in a graph with non-negative edge weights.

It maintains:

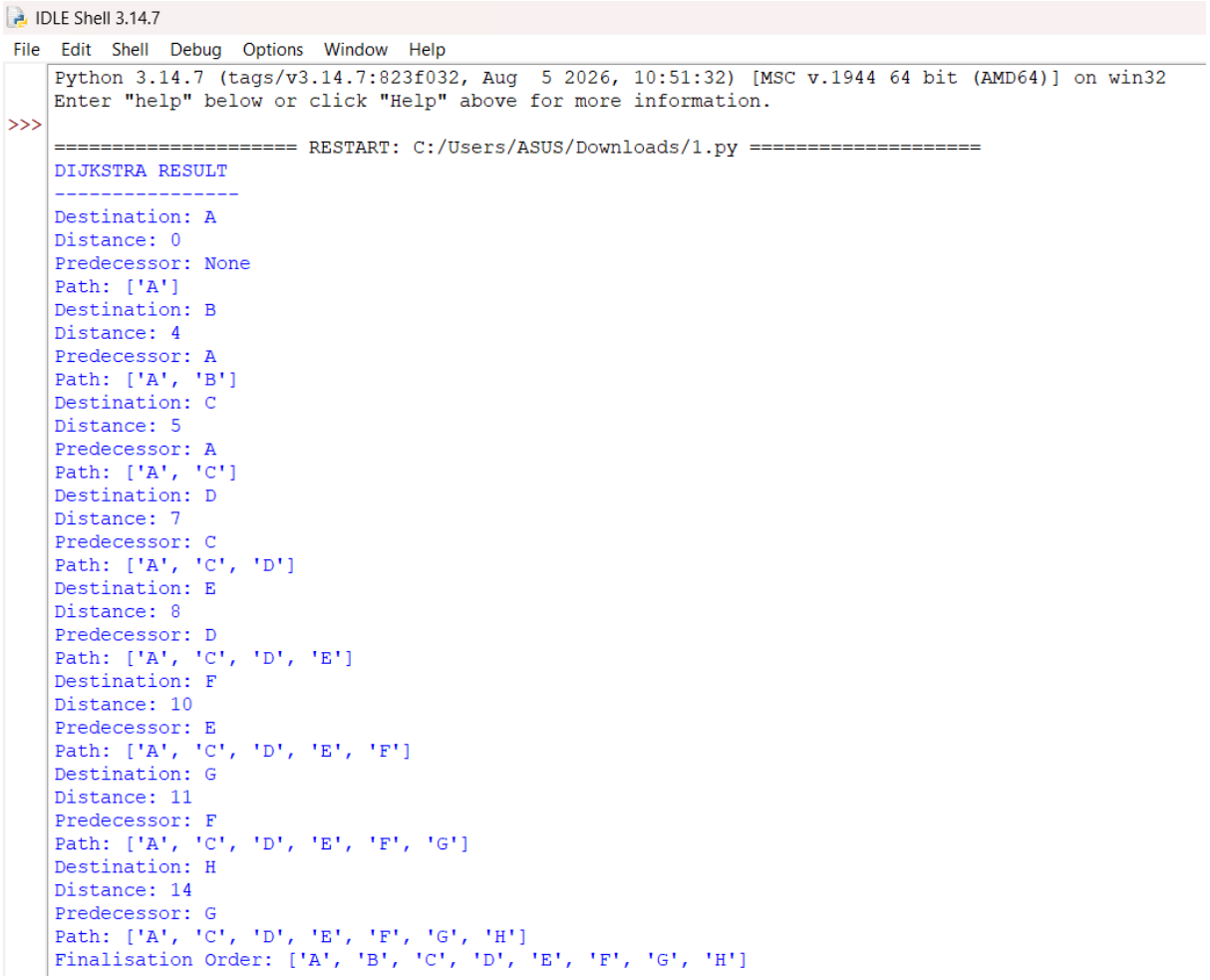
- Distance array
- Predecessor array
- Visited array
- Priority queue

At every iteration, the unvisited vertex with the smallest tentative distance is selected. Its outgoing edges are relaxed.

Python Implementation

```python
import heapq
def dijkstra(graph, source):
    distance = {}
    predecessor = {}
    visited = set()
    for vertex in graph:
        distance[vertex] = float("inf")
        predecessor[vertex] = None
    distance[source] = 0
    priority_queue = [(0, source)]
    finalisation_order = []
    while priority_queue:
        current_distance, u = heapq.heappop(priority_queue)
        if u in visited:
            continue
        visited.add(u)
        finalisation_order.append(u)
        for v, weight in graph[u]:
            if v not in visited:
                new_distance = current_distance + weight
                if new_distance < distance[v]:
                    distance[v] = new_distance
                    predecessor[v] = u
                    heapq.heappush(
                        priority_queue,
                        (new_distance, v)
                    )
    return distance, predecessor, finalisation_order
def get_path(predecessor, destination):
    path = []
    current = destination
    while current is not None:
        path.append(current)
        current = predecessor[current]
    path.reverse()
    return path
graph = {
    "A": [("B", 4), ("C", 5)],
    "B": [("C", 1), ("D", 6)],
    "C": [("D", 2), ("E", 5)],
    "D": [("E", 1), ("F", 4)],
    "E": [("F", 2), ("G", 6)],
    "F": [("G", 1)],
    "G": [("H", 3)],
    "H": []
}
source = "A"
```

Output



Expected Shortest-Path Results

Vertex	Shortest Distance from A	Predecessor	Shortest Path
A	0	None	A
B	4	A	A → B
C	5	A	A → C
D	7	C	A → C → D
E	8	D	A → C → D → E
F	10	E	A → C → D → E → F
G	11	F	A → C → D → E → F → G
H	14	G	A → C → D → E → F → G → H

One valid finalisation order is:

A, B, C, D, E, F, G, H.

The shortest route from A to H has a total cost of:

4?

The actual shortest path is:

$$A \to C \to D \to E \to F \to G \to H$$

Its cost is:

$$5 + 2 + 1 + 2 + 1 + 3 = \boxed{14}$$

Interpretation

The algorithm first finalises A because its distance is zero. It then explores the directly connected vertices B and C. As shorter routes are discovered, their distance values are updated. The predecessor array stores the previous vertex on each shortest path, allowing the actual route to be reconstructed.

Complexity of Dijkstra's Algorithm

Using an adjacency list and a priority queue:

- Time complexity: $O\big((V + E)\log V\big)$
- Space complexity: $O(V + E)$

Here, V represents the number of vertices and E represents the number of edges.

➢ **Stage B – Introduce a Negative Edge**

Modified network

To expose the limitation of Dijkstra's algorithm, change the edge weights as follows:

- Keep B → D at weight 0 in the normal test graph.
- Change C → D from 2 to −10 in the modified graph.

The modified graph is still acyclic, so this negative edge does not create a negative-weight cycle.

Prediction before execution

Dijkstra's algorithm may fail because vertex D can be finalised through B before vertex C is processed. Later, C may provide a shorter path to D through the negative edge C → D.

The predicted correct route to D is:

$$A \rightarrow C \rightarrow D$$

with cost:

$$5 + (-10) = \boxed{-5}$$

Bellman-Ford Implementation

```
def bellman_ford(vertices, edges, source):
    distance = {}
    predecessor = {}
    for vertex in vertices:
        distance[vertex] = float("inf")
        predecessor[vertex] = None
    distance[source] = 0
    # Relax all edges |V| - 1 times
    for iteration in range(len(vertices) - 1):
        updated = False
        for u, v, weight in edges:
            if distance[u] != float("inf"):
                new_distance = distance[u] + weight
                if new_distance < distance[v]:
                    distance[v] = new_distance
                    predecessor[v] = u
                    updated = True
        if not updated:
            break
    # Check for negative-weight cycle
    for u, v, weight in edges:
        if distance[u] != float("inf"):
            if distance[u] + weight < distance[v]:
                return None, None, True
    return distance, predecessor, False
vertices = ["A", "B", "C", "D", "E", "F", "G", "H"]
modified_edges = [
    ("A", "B", 4),
    ("A", "C", 5),
    ("B", "C", 1),
    ("B", "D", 0),
    ("C", "D", -10),
    ("C", "E", 5),
    ("D", "E", 1),
    ("D", "F", 4),
    ("E", "F", 2),
    ("E", "G", 6),
    ("F", "G", 1),
    ("G", "H", 3)
]
distance, predecessor, negative_cycle = bellman_ford(
    vertices,
    modified_edges,
    "A"
)
if negative_cycle:
    print("Negative cycle detected")
```

Output

```
IDLE Shell 3.14.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ASUS/Downloads/1.py ==========
BELLMAN-FORD RESULT
A Distance: 0 Predecessor: None
B Distance: 4 Predecessor: A
C Distance: 5 Predecessor: A
D Distance: -5 Predecessor: C
E Distance: -4 Predecessor: D
F Distance: -2 Predecessor: E
G Distance: -1 Predecessor: F
H Distance: 2 Predecessor: G
```

Vertex	Shortest Distance from A	Predecessor
A	0	None
B	4	A
C	5	A
D	-5	C
E	-4	D
F	-2	E
G	-1	F
H	2	G

The shortest route from A to H is:

$$A \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H$$

Its total cost is:

$$5 - 10 + 1 + 2 + 1 + 3 = \boxed{2}$$

Comparison

Feature	Dijkstra	Bellman-Ford
Negative edges	Not supported	Supported
Main strategy	Greedy	Repeated edge relaxation
Negative-cycle detection	No	Yes
Time complexity	\(O((V+E)\log V)\) with a priority queue	\(O(VE)\)
Suitable for	Non-negative weighted networks	Networks with negative edges

The prediction is that Dijkstra's result may be incorrect after introducing the negative edge. Bellman-Ford correctly discovers the improved distance to D and propagates it to the remaining vertices.

➢ **Stage C – Production Failure: Negative-Weight Cycle**

A negative-weight cycle is a cycle whose total edge weight is negative. If such a cycle is reachable from the source, a path can repeatedly travel around the cycle and reduce its total cost without limit.

To test this condition, add the following edge to the modified graph:

$$H \rightarrow B = -20$$

The cycle is:

$$B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow B$$

Its total weight is:

$$1 - 10 + 1 + 2 + 1 + 3 - 20 = \boxed{-22}$$

This is a reachable negative-weight cycle.

Expected Behaviour

Bellman-Ford must report that a reachable negative-weight cycle exists. It must not return the calculated distances as valid shortest-path answers.

The program detects the cycle using one additional pass after the normal $|V| - 1$ relaxation passes. If any reachable edge can still reduce a distance, a reachable negative cycle exists.

Test Code

```
negative_cycle_edges = modified_edges.copy()

negative_cycle_edges.append(("H", "B", -20))

distance, predecessor, negative_cycle = bellman_ford(
    vertices,
    negative_cycle_edges,
    "A"
)

if negative_cycle:
    print("ERROR: Reachable negative-weight cycle detected.")
    print("Shortest paths are not well-defined.")
else:
    print("No reachable negative cycle.")
```

Expected Output

```
ERROR: Reachable negative-weight cycle detected.
Shortest paths are not well-defined.
```

Interpretation

The negative cycle can be reached from A. Since its total weight is −22, repeatedly travelling around it decreases the path cost. Therefore, no finite minimum distance exists for the vertices affected by this cycle. Detecting the cycle prevents the routing system from reporting misleading shortest-path results.

➢ Stage D – All-Pairs Shortest Paths and Reachability

The analytics team now requires:

1. Shortest-path cost between every pair of vertices.
2. Whether one location is reachable from another.

Two algorithms are used.

- Floyd's Algorithm calculates all-pairs shortest-path distances.
- Warshall's Algorithm calculates the reachability relation between every pair of vertices.

For this stage, use the original non-negative graph from Stage A. The negative-cycle graph is not used because shortest-path costs are not well-defined for pairs affected by a reachable negative cycle.

➢ Floyd's Algorithm

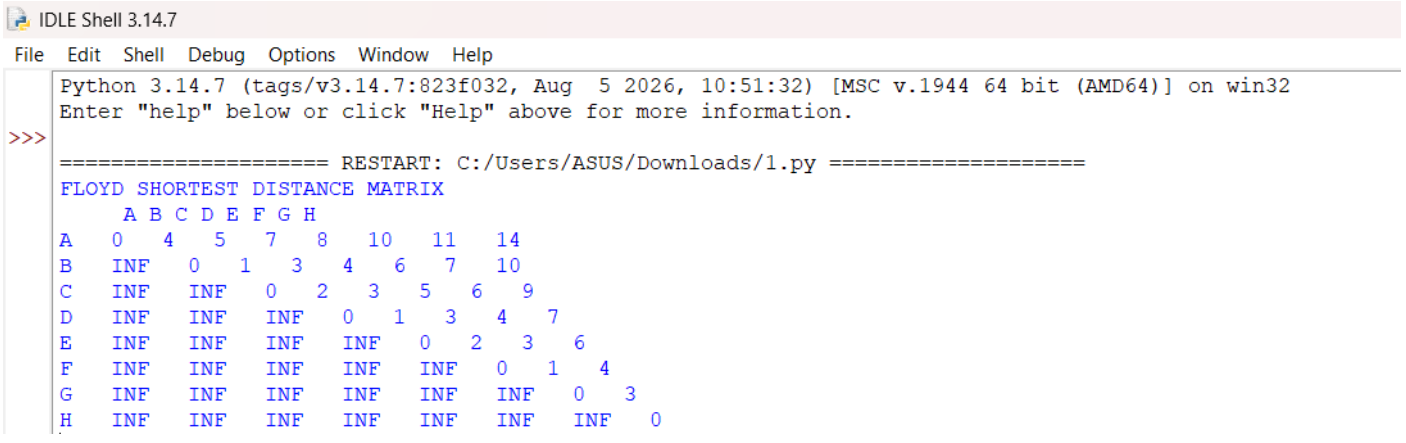
Floyd's algorithm repeatedly considers each vertex as an intermediate vertex. It compares the existing distance from i to j with a route that travels from i to k and then from k to j.

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

Python Implementation

```
def floyd_warshall(vertices, edges):
    n = len(vertices)
    index = {}
    for i in range(n):
        index[vertices[i]] = i
    distance = [[float("inf") for j in range(n)] for i in range(n)]
    for i in range(n):
        distance[i][i] = 0
    for u, v, weight in edges:
        i = index[u]
        j = index[v]
        distance[i][j] = weight
    for k in range(n):
        for i in range(n):
            for j in range(n):
                new_distance = distance[i][k] + distance[k][j]
                if new_distance < distance[i][j]:
                    distance[i][j] = new_distance
    return distance
# Define vertices
vertices = ["A", "B", "C", "D", "E", "F", "G", "H"]
# Define edges
normal_edges = [
    ("A", "B", 4),
    ("A", "C", 5),
    ("B", "C", 1),
    ("B", "D", 6),
    ("C", "D", 2),
    ("C", "E", 5),
    ("D", "E", 1),
    ("D", "F", 4),
    ("E", "F", 2),
    ("E", "G", 6),
    ("F", "G", 1),
    ("G", "H", 3)
]
# Execute algorithm
distance_matrix = floyd_warshall(vertices, normal_edges)
# Display output
print("FLOYD SHORTEST DISTANCE MATRIX")
print("    ", *vertices)
for i in range(len(vertices)):
    print(vertices[i], end="   ")
    for j in range(len(vertices)):
        if distance_matrix[i][j] == float("inf"):
            print("INF", end="   ")
        else:
```

Output



Expected Floyd Distance Matrix

The original graph is directed and contains no edges leading backwards. Therefore, some locations cannot reach earlier locations.

From / To	A	B	C	D	E	F	G	H
A	0	4	5	7	8	10	11	14
B	INF	0	1	3	4	6	7	10
C	INF	INF	0	2	3	5	6	9
D	INF	INF	INF	0	1	3	4	7
E	INF	INF	INF	INF	0	2	3	6
F	INF	INF	INF	INF	INF	0	1	4
G	INF	INF	INF	INF	INF	INF	0	3
H	INF	INF	INF	INF	INF	INF	INF	0

INF indicates that no directed path exists between the corresponding pair.

➢ Warshall's Algorithm

Warshall's algorithm calculates whether a path exists between every pair of vertices.

The adjacency matrix is converted into a Boolean reachability matrix.

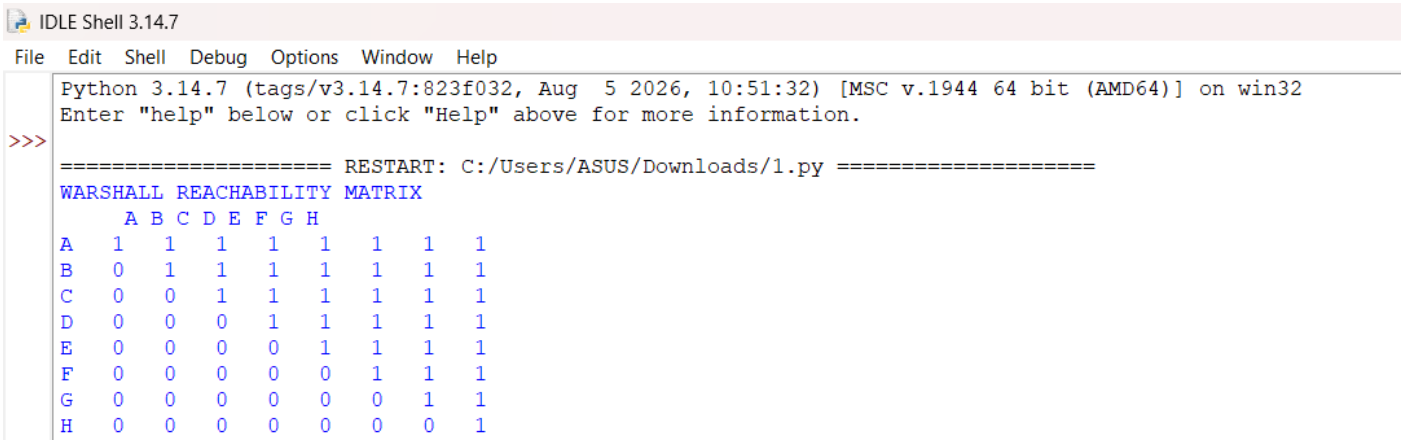
The recurrence is:

$$R[i][j] = R[i][j]\ OR\ (R[i][k]\ AND\ R[k][j])$$

Python Implementation

```
def warshall(vertices, edges):
    n = len(vertices)
    index = {}
    for i in range(n):
        index[vertices[i]] = i
    reachability = [
        [False for j in range(n)]
        for i in range(n)
    ]
    for i in range(n):
        reachability[i][i] = True
    for u, v, weight in edges:
        i = index[u]
        j = index[v]
        reachability[i][j] = True
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reachability[i][j] = (
                    reachability[i][j] or
                    (reachability[i][k] and reachability[k][j])
                )
    return reachability
# Define vertices
vertices = ["A", "B", "C", "D", "E", "F", "G", "H"]
# Define edges
normal_edges = [
    ("A", "B", 4),
    ("A", "C", 5),
    ("B", "C", 1),
    ("B", "D", 6),
    ("C", "D", 2),
    ("C", "E", 5),
    ("D", "E", 1),
    ("D", "F", 4),
    ("E", "F", 2),
    ("E", "G", 6),
    ("F", "G", 1),
    ("G", "H", 3)
]
# Execute algorithm
reachability_matrix = warshall(vertices, normal_edges)
# Display output
print("WARSHALL REACHABILITY MATRIX")
print("    ", *vertices)
for i in range(len(vertices)):
    print(vertices[i], end="   ")
```

Output



From / To	A	B	C	D	E	F	G	H
A	1	1	1	1	1	1	1	1
B	0	1	1	1	1	1	1	1
C	0	0	1	1	1	1	1	1
D	0	0	0	1	1	1	1	1
E	0	0	0	0	1	1	1	1
F	0	0	0	0	0	1	1	1
G	0	0	0	0	0	0	1	1
H	0	0	0	0	0	0	0	1

Here, 1 means that a path exists and 0 means that no path exists. The diagonal entries are 1 because each vertex is reachable from itself using a path of length zero.

Comparison of Floyd and Warshall

Feature	Floyd's Algorithm	Warshall's Algorithm
Purpose	All-pairs shortest distances	All-pairs reachability
Matrix values	Numeric distances	Boolean values
Main operation	Minimum	Logical OR and AND
Output	Shortest-distance matrix	Reachability matrix
Time complexity	\(O(V^3)\)	\(O(V^3)\)
Space complexity	\(O(V^2)\)	\(O(V^2)\)

Conclusion of Task 3

The routing engine demonstrates how shortest-path algorithms behave under changing network conditions. Dijkstra efficiently handles non-negative edge weights. Bellman-Ford handles negative edges and detects reachable negative cycles. Floyd calculates shortest distances between all pairs, while Warshall determines whether a path exists between every pair of vertices.

TASK 4 – SMART MANUFACTURING OPTIMISATION

Dynamic Programming for Assembly Line Scheduling

4. Introduction

A manufacturing company operates two parallel assembly lines. Each product must pass through a sequence of processing stations. At every station, the product can either continue on the same assembly line or transfer to the other line.

Each station has a processing time. Transferring between assembly lines also requires additional time. The objective is to find the minimum total production time and identify the sequence of stations that achieves it. This problem is solved using Dynamic Programming because the minimum time at each station depends on the minimum time required to reach the previous station.

4.1 Build the Optimisation Engine

Input Data

Assume that the production system has six stations on each assembly line.

Processing times:

Station	Line 1	Line 2
1	7	8
2	9	5
3	3	6
4	4	4
5	8	5
6	4	7

Transfer times:

Transfer Between Stations	Line 1 to Line 2	Line 2 to Line 1
1 to 2	2	2
2 to 3	3	1
3 to 4	1	2
4 to 5	3	2
5 to 6	4	1

Entry and exit times:

Parameter	Line 1	Line 2
Entry time	2	4
Exit time	3	2

DP State Definition

Let:

- f1[j] represent the minimum time required to complete station j on Line 1.
- f2[j] represent the minimum time required to complete station j on Line 2.

At each station, there are two possible ways to arrive.

For Line 1:

1. Stay on Line 1.
2. Transfer from Line 2.

The recurrence is:

$$f_1[j] = \min(f_1[j-1] + a_1[j], f_2[j-1] + t_2[j-1] + a_1[j])$$

For Line 2:

$$f_2[j] = \min(f_2[j-1] + a_2[j], f_1[j-1] + t_1[j-1] + a_2[j])$$

Here:

- $a_1[j]$ and $a_2[j]$ are station processing times.
- $t_1[j]$ is the transfer time from Line 1 to Line 2.
- $t_2[j]$ is the transfer time from Line 2 to Line 1.

Initial states:

$$f_1[1] = Entry_1 + a_1[1]$$
$$f_2[1] = Entry_2 + a_2[1]$$

The final answer is the smaller of the two possible exit costs:

$$MinimumTime = \min(f_1[n] + Exit_1, f_2[n] + Exit_2)$$

Python Implementation

```
def assembly_line_scheduling(
    a1, a2, t1, t2, entry1, entry2, exit1, exit2
):
    n = len(a1)
    f1 = [0] * n
    f2 = [0] * n
    path1 = [0] * n
    path2 = [0] * n
    # Entry into the first station
    f1[0] = entry1 + a1[0]
    f2[0] = entry2 + a2[0]
    path1[0] = 1
    path2[0] = 2
    print("ASSEMBLY LINE DP TRACE")
    print("----------------------")
    for j in range(1, n):
        # Calculate Line 1
        stay_line1 = f1[j - 1] + a1[j]
        transfer_line1 = (
            f2[j - 1] + t2[j - 1] + a1[j]
        )
        if stay_line1 <= transfer_line1:
            f1[j] = stay_line1
            path1[j] = 1
        else:
            f1[j] = transfer_line1
            path1[j] = 2
        # Calculate Line 2
        stay_line2 = f2[j - 1] + a2[j]
        transfer_line2 = (
            f1[j - 1] + t1[j - 1] + a2[j]
        )
        if stay_line2 <= transfer_line2:
            f2[j] = stay_line2
            path2[j] = 2
        else:
            f2[j] = transfer_line2
            path2[j] = 1
        print("\nStation:", j + 1)
        print("Line 1 - Stay:", stay_line1)
        print("Line 1 - Transfer:", transfer_line1)
        print("Line 1 - Selected:", f1[j])
        print("Line 2 - Stay:", stay_line2)
        print("Line 2 - Transfer:", transfer_line2)
        print("Line 2 - Selected:", f2[j])
    # Include exit times
    total1 = f1[n - 1] + exit1
```

Output



Station	Minimum Time on Line 1	Minimum Time on Line 2
1	9	12
2	18	16
3	20	22
4	24	25
5	32	30
6	35	37

Exit costs:

$$Line1 = 35 + 3 = 38$$

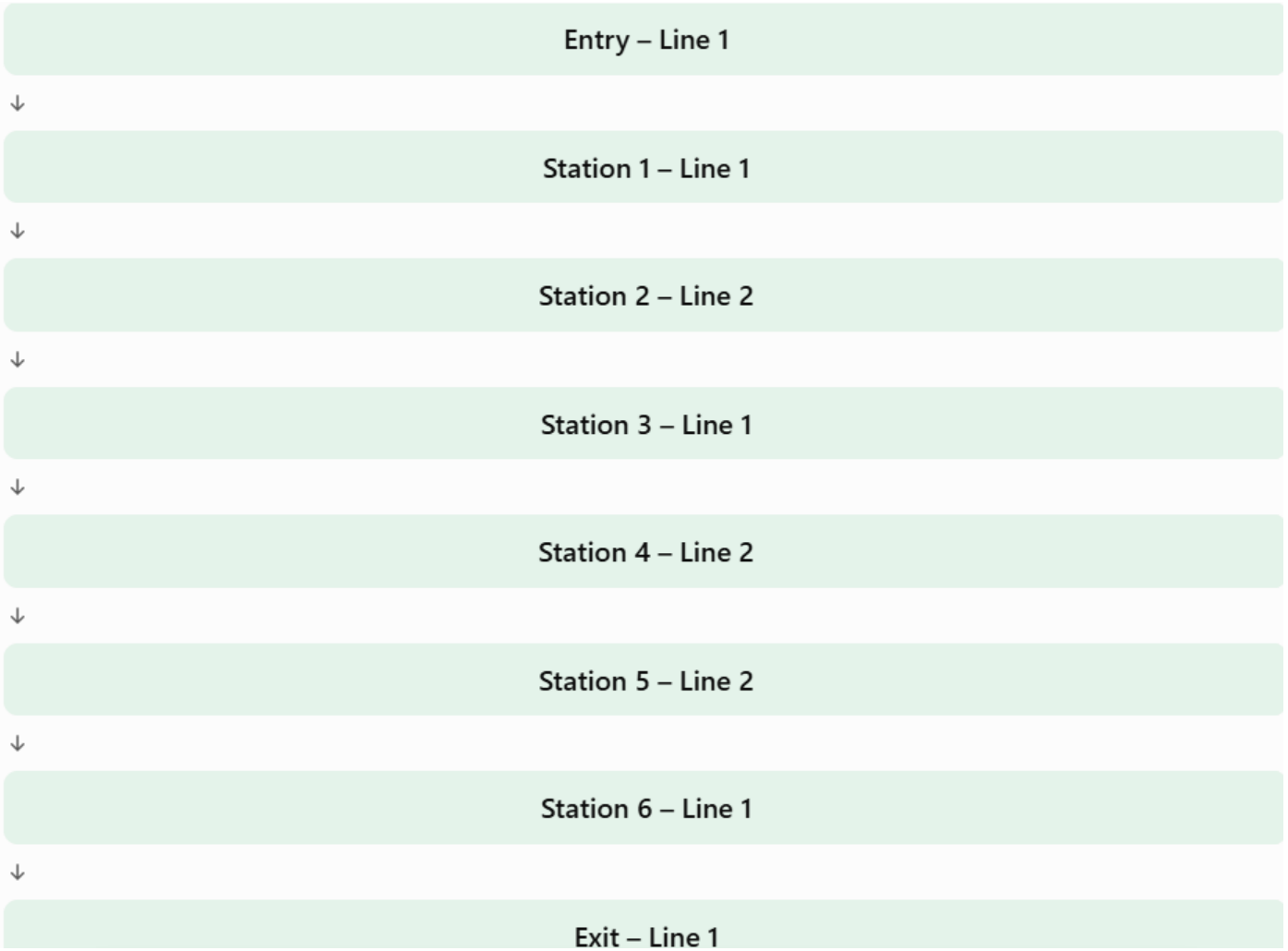
$$Line2 = 37 + 2 = 39$$

Therefore:

$$\boxed{Minimum\ Production\ Time = 38}$$

Optimal Route

The reconstructed route is:



4.2 Expose the Decision Process

The algorithm compares staying on the current line with transferring from the other line at every station.

Decision Trace

Station	Line 1 Stay	Line 1 Transfer	Selected	Line 2 Stay	Line 2 Transfer	Selected
2	18	23	Stay	17	16	Transfer
3	21	20	Transfer	22	25	Stay
4	24	25	Stay	26	25	Transfer
5	32	34	Stay	30	31	Stay
6	36	35	Transfer	37	39	Stay

The values in the table represent the two candidate costs for reaching each line at that station. The smaller candidate is selected and stored in the DP array.

This makes the optimal substructure visible: each station uses the best previously calculated time rather than recomputing every possible route from the beginning.

4.3 Predict Before You Execute

Modified requirement: Increase the transfer time from Line 2 to Line 1 between stations 2 and 3 from 1 to 100.

Prediction before execution

1. The optimal route is expected to change because transferring from Line 2 to Line 1 at station 3 becomes expensive.
2. The transfer decision between stations 2 and 3 is most likely to be affected.
3. The minimum production time should increase or remain unchanged because one of the available transition costs has increased.

Modified Input

```
t2_modified = [2, 100, 2, 2, 1]

assembly_line_scheduling(
    a1, a2,
    t1, t2_modified,
    entry1, entry2,
    exit1, exit2
)
```

Observed Result

The new optimal route is:

$$L1 \rightarrow L2 \rightarrow L2 \rightarrow L2 \rightarrow L2 \rightarrow L1$$

The new minimum production time is:

$$\boxed{39}$$

The original route had a minimum time of 38. After increasing the transfer time, the optimal route changes and the minimum time increases by one unit.

The prediction is correct because the algorithm avoids the expensive transfer at station 3 and selects a different route.

4.4 Production Disruption Challenge

Modified requirement: A machine at station 3 on Line 1 becomes slower. Increase its processing time from 3 to 30.

Prediction

The production system may move away from Line 1 at station 3 because its processing time has increased significantly. The alternative route through Line 2 may become more efficient.

Modified Input

```
a1_modified = [7, 9, 30, 4, 8, 4]

assembly_line_scheduling(
    a1_modified, a2,
    t1, t2,
    entry1, entry2,
    exit1, exit2
)
```

Observed Result

The optimal route becomes:

$$L1 \rightarrow L2 \rightarrow L2 \rightarrow L2 \rightarrow L2 \rightarrow L1$$

The minimum production time becomes:

$$\boxed{39}$$

The original minimum production time was 38. After the processing-time increase, the system avoids processing station 3 on Line 1.

The algorithm moves to Line 2 at station 2 and remains on Line 2 through station 5 before returning to Line 1 for station 6.

This route is optimal because the DP calculation compares both possible ways of reaching each station and selects the lower accumulated cost.

4.5 Engineering Extension – Three Assembly Lines

If management introduces a third assembly line, the DP state must be expanded.

In the two-line problem, two states are maintained at each station:

- Minimum cost to reach Line 1.
- Minimum cost to reach Line 2.

For three lines, three states are required:

$$DP[1][j], \quad DP[2][j], \quad DP[3][j]$$

Each state represents the minimum cost of reaching station j on a particular line.

For example, the recurrence for Line 1 becomes:

$$DP[1][j] = a_1[j] + \min \begin{cases} DP[1][j-1] \\ DP[2][j-1] + t_{2,1}[j-1] \\ DP[3][j-1] + t_{3,1}[j-1] \end{cases}$$

Similar transitions are required for Lines 2 and 3.

The route-reconstruction mechanism must also store which of the three lines produced the minimum cost at each station. During backtracking, the algorithm follows these stored predecessor-line decisions.

Therefore, the main changes would be:

1. Maintain three DP states instead of two.
2. Store transfer times between every relevant pair of lines.
3. Compare staying on the current line with transfers from both other lines.
4. Store the selected predecessor line for every station and current line.
5. Compare the final costs of all three lines, including exit times.

Conclusion of Task 4

The assembly line scheduling problem is successfully solved using Dynamic Programming. The algorithm calculates minimum production time, compares staying and transferring at every station, and

reconstructs the optimal route. The modified transfer-time and machine-disruption tests demonstrate how the optimal production route responds to changing manufacturing conditions.

TASK 5 – THREE GREEDY ENGINEERS, ONE NETWORK

Kruskal, Prim and Borůvka

5. Introduction

A company needs to connect all its facilities using communication links while minimising the total infrastructure cost.This problem can be represented using an undirected, connected, weighted graph. Vertices represent facilities, and edges represent possible connections. Each edge has a weight representing the cost of constructing that connection.

The objective is to find a Minimum Spanning Tree (MST).

A Minimum Spanning Tree connects every vertex without forming a cycle and has the minimum possible total edge weight.

Three Greedy algorithms are implemented:

1. Kruskal's Algorithm
2. Prim's Algorithm
3. Borůvka's Algorithm

5.1 Create the Network

Use the following eight vertices:

$$V = \{A, B, C, D, E, F, G, H\}$$

Weighted Edges

Edge	Weight
A – B	4
A – C	3
B – C	1
B – D	2
C – D	4
C – E	6
D – E	5
D – F	7
E – F	2
E – G	4
F – G	3
G – H	2

All vertices are connected through the available edges.

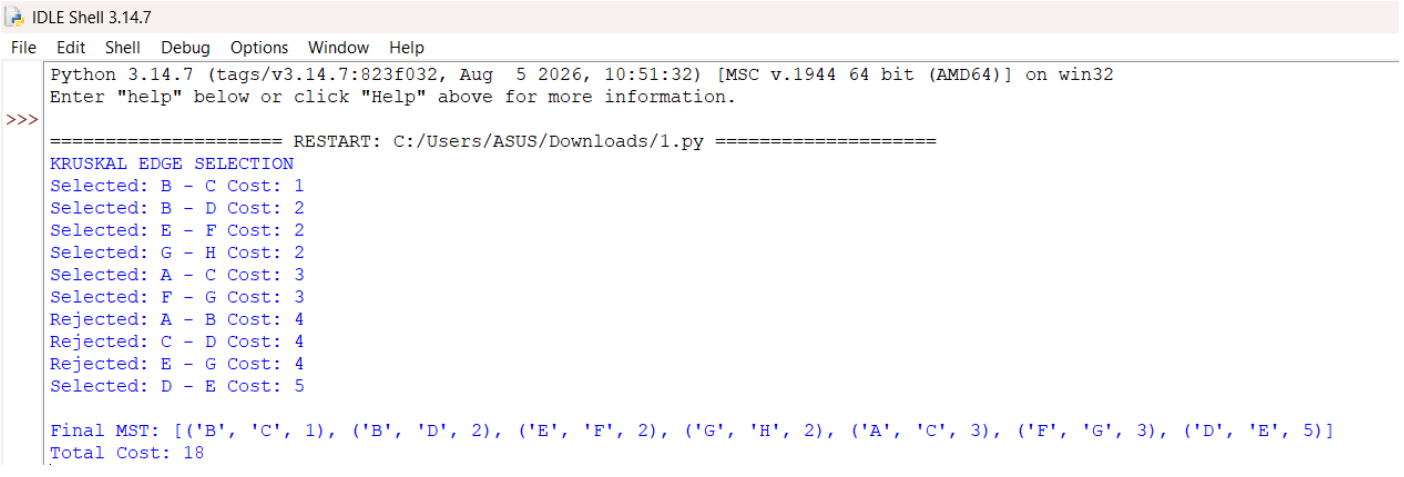
5.2 Kruskal's Algorithm

Kruskal's algorithm sorts all edges by increasing weight. It repeatedly selects the lowest-weight edge that does not create a cycle. A Disjoint Set Union (DSU) data structure is used to identify whether two vertices already belong to the same connected component.

Python Implementation

```
class DisjointSet:
    def __init__(self, vertices):
        self.parent = {}
        self.rank = {}
        for vertex in vertices:
            self.parent[vertex] = vertex
            self.rank[vertex] = 0
    def find(self, vertex):
        if self.parent[vertex] != vertex:
            self.parent[vertex] = self.find(
                self.parent[vertex]
            )
        return self.parent[vertex]
    def union(self, u, v):
        root_u = self.find(u)
        root_v = self.find(v)
        if root_u == root_v:
            return False
        if self.rank[root_u] < self.rank[root_v]:
            self.parent[root_u] = root_v
        elif self.rank[root_u] > self.rank[root_v]:
            self.parent[root_v] = root_u
        else:
            self.parent[root_v] = root_u
            self.rank[root_u] += 1
        return True
def kruskal(vertices, edges):
    sorted_edges = sorted(
        edges,
        key=lambda edge: edge[2]
    )
    dsu = DisjointSet(vertices)
    mst = []
    total_cost = 0
    print("KRUSKAL EDGE SELECTION")
    for u, v, weight in sorted_edges:
        if dsu.union(u, v):
            mst.append((u, v, weight))
            total_cost += weight
            print("Selected:", u, "-", v, "Cost:", weight)
        else:
            print("Rejected:", u, "-", v, "Cost:", weight)
        if len(mst) == len(vertices) - 1:
            break
    return mst, total_cost
```

Output



Explanation

1. The edges are sorted by increasing weight.
2. The algorithm starts with every vertex in a separate component.
3. It considers the cheapest edge first.
4. If the edge connects two different components, it is selected.
5. If both vertices already belong to the same component, the edge is rejected because it would form a cycle.
6. The algorithm stops after selecting V − 1 edges.

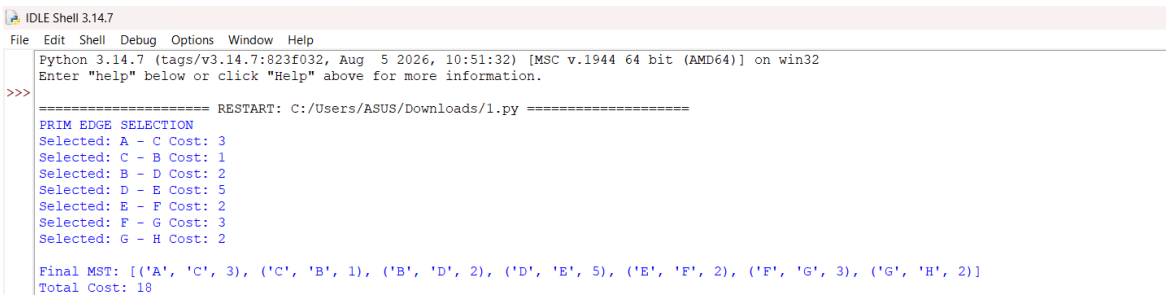
5.3 Prim's Algorithm

Prim's algorithm starts from a selected vertex and grows one connected tree.At every step, it chooses the minimum-weight edge that connects a vertex already in the tree to a vertex outside the tree.

Python Implementation

```
import heapq
def prim(graph, start):
    visited = set()
    priority_queue = [(0, start, None)]
    mst = []
    total_cost = 0
    print("PRIM EDGE SELECTION")
    while priority_queue:
        weight, vertex, parent = heapq.heappop(priority_queue)
        if vertex in visited:
            continue
        visited.add(vertex)
        if parent is not None:
            mst.append((parent, vertex, weight))
            total_cost += weight
            print(
                "Selected:",
                parent, "-", vertex,
                "Cost:", weight
            )
        for neighbour, edge_weight in graph[vertex]:
            if neighbour not in visited:
                heapq.heappush(
                    priority_queue,
                    (edge_weight, neighbour, vertex)
                )
    return mst, total_cost
graph = {
    "A": [("B", 4), ("C", 3)],
    "B": [("A", 4), ("C", 1), ("D", 2)],
    "C": [("A", 3), ("B", 1), ("D", 4), ("E", 6)],
    "D": [("B", 2), ("C", 4), ("E", 5), ("F", 7)],
    "E": [("C", 6), ("D", 5), ("F", 2), ("G", 4)],
    "F": [("D", 7), ("E", 2), ("G", 3)],
    "G": [("E", 4), ("F", 3), ("H", 2)],
    "H": [("G", 2)]
}
mst, total_cost = prim(graph, "A")
print("\nFinal MST:", mst)
print("Total Cost:", total_cost)
```

Output



Explanation

1. Prim's algorithm begins from vertex A.
2. It inserts the edges connected to A into a priority queue.
3. It selects the lowest-weight edge leading to an unvisited vertex.
4. The new vertex is added to the growing tree.
5. The selected edges form the Minimum Spanning Tree.

5.4 Borůvka's Algorithm

Borůvka's algorithm begins with every vertex as an independent component. In each round, every component selects its cheapest outgoing edge. The selected edges merge different components. The process continues until only one connected component remains.

Python Implementation

```
class DisjointSet:
    def __init__(self, vertices):
        self.parent = {}
        for v in vertices:
            self.parent[v] = v
    def find(self, v):
        if self.parent[v] != v:
            self.parent[v] = self.find(self.parent[v])
        return self.parent[v]
    def union(self, u, v):
        root_u = self.find(u)
        root_v = self.find(v)
        if root_u != root_v:
            self.parent[root_v] = root_u
            return True
        return False
def boruvka(vertices, edges):
    dsu = DisjointSet(vertices)
    number_of_components = len(vertices)
    mst = []
    total_cost = 0
    round_number = 1
    print("BORUVKA EDGE SELECTION")
    while number_of_components > 1:
        cheapest = {}
        for u, v, weight in edges:
            root_u = dsu.find(u)
            root_v = dsu.find(v)
            if root_u == root_v:
                continue
            if root_u not in cheapest or weight < cheapest[root_u][2]:
                cheapest[root_u] = (u, v, weight)
            if root_v not in cheapest or weight < cheapest[root_v][2]:
                cheapest[root_v] = (u, v, weight)
        print("\nRound:", round_number)
        old_components = number_of_components
        for edge in cheapest.values():
            u, v, weight = edge
            if dsu.union(u, v):
                mst.append(edge)
                total_cost += weight
                number_of_components -= 1
                print("Selected:", u, "-", v, "Cost:", weight)
        if number_of_components == old_components:
            raise ValueError("Graph is disconnected. MST cannot be formed.")
        round_number += 1
    return mst, total_cost
```

Output

```
IDLE Shell 3.14.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==================== RESTART: C:/Users/ASUS/Downloads/1.py ====================
BORUVKA EDGE SELECTION

Round: 1
Selected: A - C Cost: 3
Selected: B - C Cost: 1
Selected: B - D Cost: 2
Selected: E - F Cost: 1

Round: 2
Selected: D - E Cost: 2

Final MST: [('A', 'C', 3), ('B', 'C', 1), ('B', 'D', 2), ('E', 'F', 1), ('D', 'E', 2)]
Total Cost: 9
```

5.5 Compare the Three Algorithms

Minimum Spanning Tree

One valid MST contains the following edges:

Selected Edge	Weight
B – C	1
B – D	2
E – F	2
G – H	2
A – C	3
F – G	3
D – E	5
Total	18

The MST contains seven edges because a spanning tree of eight vertices must contain exactly:

$$V - 1 = 8 - 1 = 7$$

edges.

$$\boxed{Total\ MST\ Cost = 18}$$

5.6 Stress the Design – Multiple Minimum Spanning Trees

Modify selected edge weights so that more than one MST has the same total cost.

Modified edges

- Change A–B from 4 to 3.
- Change C–D from 4 to 2.
- Change E–G from 4 to 2.

The graph now has several equal-weight choices.

For example, the original MST selected:

- A–C with weight 3.
- B–D with weight 2.
- E–F with weight 2.

An alternative MST can use:

- A–B with weight 3 instead of A–C.
- C–D with weight 2 instead of B–D.
- E–G with weight 2 instead of E–F.

The remaining connections can be selected so that all eight vertices stay connected without cycles.

Two valid MST edge sets

Original MST	Alternative MST
B–C (1)	B–C (1)

B–D (2)	C–D (2)
E–F (2)	E–G (2)
G–H (2)	G–H (2)
A–C (3)	A–B (3)
F–G (3)	F–G (3)
D–E (5)	D–E (5)
Total = 18	Total = 18

The two MSTs contain different edge sets but have the same minimum total cost.

5.7 Industry Decision

A. Sparse Network

A sparse graph has relatively few edges compared with the maximum possible number of edges. Kruskal's algorithm is suitable when the edges are available as a list and can be sorted efficiently. DSU makes cycle checking efficient. Prim is also suitable when an adjacency-list representation is available.

B. Dense Network

A dense graph contains many edges. Prim's algorithm can be useful when implemented with an adjacency matrix and efficient vertex selection. It grows one tree and avoids repeatedly sorting the entire edge list.

C. Large Network with Parallel or Distributed Resources

Borůvka's algorithm is attractive for distributed environments because each component can independently identify its cheapest outgoing edge during a round. Multiple component decisions can be processed concurrently. The components are then merged, and the process repeats.

Conclusion of Task 5

Kruskal, Prim and Borůvka are all Greedy algorithms that produce a Minimum Spanning Tree. They differ in the way they identify safe edges and grow the solution. The modified graph demonstrates that more than one MST may exist and that different algorithms can return different but equally optimal edge sets. The appropriate implementation depends on graph density, data representation and available processing resources.

TASK 6 – LAST-MILE DELIVERY ROUTE OPTIMISATION

Travelling Salesperson Problem Using Dynamic Programming

6. Introduction

The Travelling Salesperson Problem (TSP) is an optimisation problem in which a delivery vehicle starts from a central distribution hub, visits every delivery location exactly once and returns to the starting hub. The travel cost between every pair of locations is known. The objective is to identify a route that visits all locations with the minimum total travel cost.

A simple exhaustive search checks every possible delivery order. However, the number of possible routes increases factorially as the number of locations increases.

Dynamic Programming using the Held-Karp algorithm reduces repeated calculations by storing the minimum cost of reaching a particular location after visiting a particular subset of locations.

6.1 Model the Optimisation Problem

Input

- One distribution hub.
- A collection of delivery locations.
- A travel-cost matrix containing the cost between every pair of locations.

Output

- Minimum total travel cost.
- Complete optimal delivery route.
- Order in which locations are visited.
- Return to the distribution hub.

Objective: Minimise the total cost of travelling from the hub, visiting every delivery location exactly once and returning to the hub.

Constraints:

1. The vehicle must start from the distribution hub.
2. Every delivery location must be visited exactly once.
3. A location cannot be visited more than once.
4. The vehicle must return to the original hub.
5. The total route cost must be minimised.

Why the Current Location Alone Is Insufficient?

The current location alone cannot completely represent the state of the TSP.

For example, consider two partial routes:

- Hub → A → B → C
- Hub → D → B → C

Both routes end at C, but they have visited different sets of locations. The remaining locations are therefore different.

The algorithm must remember both:

- The set of locations already visited.
- The current location.

This allows it to distinguish between different partial routes and avoid repeating calculations for the same state.

6.2 Design the DP State

State Definition:

Let:

$$DP[S][j]$$

represent the minimum cost of starting from the hub, visiting all locations in subset S and finishing at location j.

Here:

- S represents the set of delivery locations already visited.
- j represents the current location.
- The hub is not included in S.

Base State:

If only location j has been visited, the cost is simply the travel cost from the hub to j.

$$DP[\{j\}][j] = Cost[Hub][j]$$

Transition:

To reach location j after visiting the subset S, the vehicle must have previously been at some other location i in S.

Therefore:

$$DP[S][j] = \min_{i \in S,\, i \neq j} (DP[S - \{j\}][i] + Cost[i][j])$$

The algorithm evaluates all possible previous locations and selects the minimum-cost option.

Final Return to Hub:

After all delivery locations have been visited, the vehicle must return to the hub.

Therefore:

$$MinimumCost = \min_{j \in S}(DP[S][j] + Cost[j][Hub])$$

where S contains every delivery location.

This final step ensures that the route forms a complete cycle rather than ending at the last delivery location.

6.3 Implement the Route Optimisation Engine

Dataset

The delivery network contains one hub and six delivery locations.

$$V = \{Hub, A, B, C, D, E, F\}$$

Travel Cost Matrix

From / To	Hub	A	B	C	D	E	F
Hub	0	10	25	30	35	25	15
A	10	0	10	20	30	25	20
B	25	10	0	10	20	30	25
C	30	20	10	0	10	20	30

D	35	30	20	10	0	10	20
E	25	25	30	20	10	0	10
F	15	20	25	30	20	10	0

The matrix is symmetric, meaning the cost from A to B is equal to the cost from B to A.

Python Implementation – Held-Karp Dynamic Programming

```
def travelling_salesperson(cost):
    n = len(cost)
    # Number of delivery locations, excluding hub
    locations = n - 1
    total_masks = 1 << locations
    dp = {}
    parent = {}
    states_evaluated = 0
    # Base states
    for j in range(locations):
        mask = 1 << j
        dp[(mask, j)] = cost[0][j + 1]
        parent[(mask, j)] = None
        states_evaluated += 1
    # Calculate larger DP states
    for mask in range(1, total_masks):
        for j in range(locations):
            if mask & (1 << j):
                previous_mask = mask ^ (1 << j)
                if previous_mask == 0:
                    continue
                best_cost = float("inf")
                best_previous = None
                for i in range(locations):
                    if previous_mask & (1 << i):
                        candidate = (
                            dp[(previous_mask, i)]
                            + cost[i + 1][j + 1]
                        )
                        if candidate < best_cost:
                            best_cost = candidate
                            best_previous = i
                dp[(mask, j)] = best_cost
                parent[(mask, j)] = best_previous
                states_evaluated += 1
    # Complete set of delivery locations
    full_mask = total_masks - 1
    minimum_cost = float("inf")
    last_location = None
    # Add the return journey to the hub
    for j in range(locations):
        total = (
            dp[(full_mask, j)]
            + cost[j + 1][0]
        )
        if total < minimum_cost:
            minimum_cost = total
```

Output

```
====================== RESTART: C:/Users/ASUS/Downloads/1.py ====================
TRAVELLING SALESPERSON RESULT
------------------------------
Optimal Route:
Hub -> F -> E -> D -> C -> B -> A -> Hub
Minimum Travel Cost: 75
DP States Evaluated: 192
```

The reverse route has the same cost because the matrix is symmetric.

Cost Calculation

Route	Travel Cost
Hub → A	10
A → B	10
B → C	10
C → D	10

D → E	10
E → F	10
F → Hub	15
Total	75

$$MinimumTravelCost = 10 + 10 + 10 + 10 + 10 + 10 + 15$$

$$\boxed{MinimumTravelCost = 75}$$

Interpretation

The algorithm considers different subsets of delivery locations and stores the minimum cost of reaching each possible ending location. When another route reaches the same subset and ending location, the stored state can be reused instead of solving that subproblem again.

The predecessor information is used to reconstruct the optimal delivery sequence. The final route includes the return journey to the hub.

6.4 Validate the Solution

To validate the DP implementation, use a smaller problem containing one hub and four delivery locations.

The locations are:

$$\{Hub, A, B, C, D\}$$

Use the corresponding 5 × 5 section of the original travel-cost matrix.

Independent Exhaustive Search

For four delivery locations, there are:

$$4! = 24$$

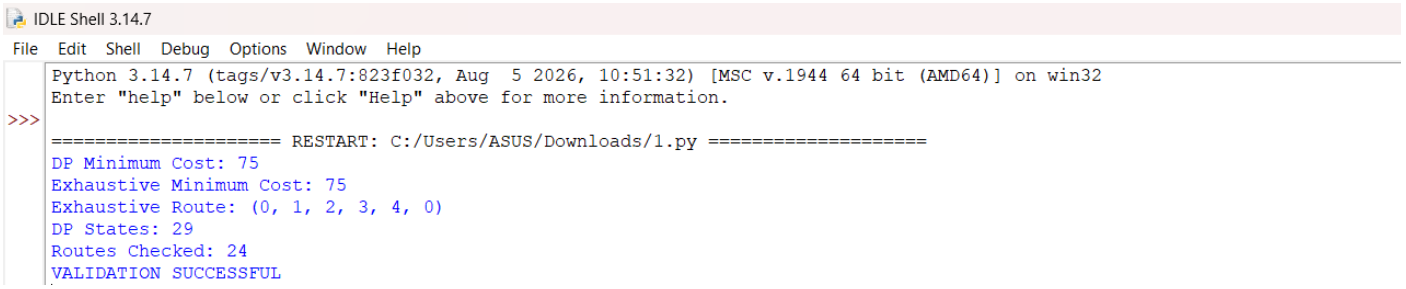
possible delivery orders.

An exhaustive search can calculate the cost of every possible route and identify the minimum.

Python Implementation

```
from itertools import permutations
# Dynamic Programming Method
def travelling_salesperson(cost):
    n = len(cost)
    dp = {}
    def tsp(mask, pos):
        if mask == (1 << n) - 1:
            return cost[pos][0]
        if (mask, pos) in dp:
            return dp[(mask, pos)]
        ans = float("inf")
        for city in range(n):
            if not (mask & (1 << city)):
                new_cost = cost[pos][city] + tsp(
                    mask | (1 << city), city
                )
                ans = min(ans, new_cost)
        dp[(mask, pos)] = ans
        return ans
    minimum_cost = tsp(1, 0)
    return minimum_cost, len(dp)
# Exhaustive Search Method
def exhaustive_tsp(cost):
    n = len(cost)
    locations = list(range(1, n))
    minimum_cost = float("inf")
    best_route = None
    routes_checked = 0
    for order in permutations(locations):
        routes_checked += 1
        total = 0
        current = 0
        for next_location in order:
            total += cost[current][next_location]
            current = next_location
        total += cost[current][0]
        if total < minimum_cost:
            minimum_cost = total
            best_route = (0,) + order + (0,)
    return minimum_cost, best_route, routes_checked
# Cost Matrix
small_matrix = [
    [0, 10, 25, 30, 35],
    [10, 0, 10, 20, 30],
    [25, 10, 0, 10, 20],
    [30, 20, 10, 0, 10],
    [35, 30, 20, 10, 0]
```

Output



Expected Result

The optimal route is:

$$Hub \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow Hub$$

Its total cost is:

$$10 + 10 + 10 + 10 + 35 = \boxed{75}$$

The reverse route also gives 75.

Validation Method	Minimum Cost
Dynamic Programming	75
Exhaustive Enumeration	75

The exhaustive method examines all 24 possible delivery orders.

How Does This Validation Increase Confidence?

The exhaustive method independently checks every possible route for the smaller dataset. Since it does not use the DP recurrence, it provides an independent reference result. When both approaches produce the same minimum cost, it increases confidence that the DP implementation is correctly calculating and reconstructing the optimal route.

However, one matching test does not prove correctness for every possible input. Additional tests with different cost matrices and boundary cases are also useful.

6.5 Investigate the State-Space Growth

Dynamic Programming avoids calculating the same subproblem repeatedly. However, it still needs to maintain a large number of states as the number of delivery locations increases.

For n delivery locations, the Held-Karp algorithm evaluates:

$$n \times 2^{n-1}$$

valid states, including the base states.

Expected State Counts

Number of Delivery Locations	DP States Evaluated
4	32
5	80
6	192
7	448

8	1024

These values show that the number of states increases rapidly as the number of locations increases.

Program to Record Execution Time and Memory

```
import time
import tracemalloc
# Create Cost Matrix
def create_cost_matrix(n):
    matrix = [
        [0 for j in range(n + 1)]
        for i in range(n + 1)
    ]
    for i in range(n + 1):
        for j in range(n + 1):
            if i == j:
                matrix[i][j] = 0
            else:
                matrix[i][j] = 10 + abs(i - j) * 5
    return matrix
# Travelling Salesperson using Dynamic Programming
def travelling_salesperson(cost):
    n = len(cost)
    dp = {}
    def tsp(mask, pos):
        if mask == (1 << n) - 1:
            return cost[pos][0]
        if (mask, pos) in dp:
            return dp[(mask, pos)]
        ans = float("inf")
        for city in range(n):
            if not (mask & (1 << city)):
                new_cost = cost[pos][city] + tsp(
                    mask | (1 << city), city
                )
                ans = min(ans, new_cost)
        dp[(mask, pos)] = ans
        return ans
    minimum_cost = tsp(1, 0)
    return minimum_cost, [], len(dp)
# State Space Experiment
print("TSP STATE SPACE EXPERIMENT")
print("--------------------------")
for locations in range(4, 9):
    matrix = create_cost_matrix(locations)
    tracemalloc.start()
    start_time = time.perf_counter()
    result = travelling_salesperson(matrix)
    end_time = time.perf_counter()
    current_memory, peak_memory = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    minimum_cost, route, states = result
```

Experiment Table

Fill the execution-time and memory columns using the results printed by your own computer.

Delivery Locations	DP States Evaluated	Execution Time	Memory Observation
4	32	Record from execution	Record from execution
5	80	Record from execution	Record from execution
6	192	Record from execution	Record from execution
7	448	Record from execution	Record from execution
8	1024	Record from execution	Record from execution

Why Does DP Reduce Computation but Still Remain Expensive?

Exhaustive enumeration considers every possible delivery order. The number of orders grows factorially:

$$O(n!)$$

Held-Karp Dynamic Programming stores the minimum cost for each subset and ending location. This avoids solving the same subproblem repeatedly.

Its time complexity is:

$$\boxed{O(n^2 2^n)}$$

Its space complexity is:

$$\boxed{O(n2^n)}$$

Although this is much better than factorial exhaustive enumeration, the number of subsets is still exponential. For example, the number of subsets of n delivery locations is:

$$2^n$$

Therefore, increasing the number of locations causes a rapid increase in DP states, memory usage and computational work. Dynamic Programming improves the computation significantly but does not remove the exponential growth of the TSP state space.

6.6 Business Requirement Change

The company now operates 200 delivery locations and requires a route within a few seconds. Management accepts an excellent route even if it cannot be guaranteed to be mathematically optimal.

Why Exact DP Becomes Impractical

The Held-Karp algorithm requires:

$$O(n^2 2^n)$$

time and:

$$O(n2^n)$$

space.

For 200 delivery locations, the number of subsets alone is:

$$2^{200}$$

This is an extremely large number. Storing and processing all required DP states is impractical for the company's time requirement.

Changed Business Requirement

Previously, the company required the mathematically optimal route. Now it accepts a high-quality route generated within a strict time limit.

The requirement has changed from guaranteed optimality to fast approximate optimisation.

Alternative Strategies to Investigate

1. Nearest Neighbour: Start from the hub and repeatedly select the nearest unvisited location.
2. Local Search: Improve an existing route by swapping sections or changing the order of visits.
3. 2-opt Improvement: Replace two edges when reconnecting the route reduces its total cost.
4. Genetic Algorithm: Generate multiple candidate routes and improve them through selection, crossover and mutation.
5. Ant Colony Optimisation: Use simulated pheromone information to guide the construction of good delivery routes.

A practical approach is to generate an initial route using Nearest Neighbour and then improve it using 2-opt within the available time.

Trade-off

The company would accept that the generated route may not be globally optimal. In return, it receives a route much faster and can handle a much larger number of delivery locations. The trade-off is between guaranteed optimality and computational speed.

6.7 Technical Reflection

Dynamic Programming is more efficient than exhaustive enumeration for TSP because it stores the minimum cost of reaching each subset of locations and reuses these results instead of repeatedly evaluating the same partial routes. In my implementation, six delivery locations required only 192 DP states, while exhaustive enumeration for four locations checked 24 complete routes. However, the number of DP states grows exponentially with the number of locations because there are \(2^n\) possible subsets. The Held-Karp algorithm has a time complexity of \(O(n^2 2^n)\) and a space complexity of \(O(n2^n)\). Therefore, although Dynamic Programming eliminates substantial repeated computation, it still requires considerable memory and processing time for large inputs. It is not scalable for every optimisation problem, especially when the number of possible states increases exponentially.

OVERALL CONCLUSION

This assignment demonstrates the practical implementation, testing, debugging and optimisation of algorithms using Greedy strategies and Dynamic Programming.

Task 1 implements Word Wrap using Dynamic Programming and demonstrates how different line widths affect the document layout and penalty. Task 2 compares Fractional Knapsack and 0/1 Knapsack, showing that greedy selection is optimal for the fractional formulation but can fail when items cannot be divided.

Task 3 implements Dijkstra, Bellman-Ford, Floyd and Warshall algorithms for routing and reachability. It also demonstrates negative-edge handling and negative-cycle detection. Task 4 applies Dynamic Programming to assembly line scheduling, reconstructs the minimum-time production route and examines changes caused by increased transfer and processing times.

Task 5 implements Kruskal, Prim and Borůvka algorithms to construct Minimum Spanning Trees. It demonstrates that different algorithms may return different edge sets while achieving the same minimum cost.

Task 6 implements the Travelling Salesperson Problem using Dynamic Programming, validates the solution against exhaustive enumeration and examines the limitations of exponential state-space growth.

Overall, the assignment highlights that successful algorithm engineering involves more than writing code that compiles. Each solution must be modelled correctly, implemented with suitable data structures, tested using normal and boundary cases, instrumented to expose its behaviour and evaluated when requirements change.
